4. Go под капотом

Массивы, срезы, map, интерфейсы, каналы, select и context

arrays → slices

память, aliasing, append

maps

old buckets → Swiss Tables → sync.Map

interfaces

dynamic type, itab, typed nil

channels → select

hchan, sudog, parking

context

cancellation tree, timers, propagation

syntax → runtime representation → memory/synchronization → scheduler/GC → production symptoms

Версионные детали помечены отдельно. Текущая точка отсчета: Go 1.27.1.

МЕНТАЛЬНАЯ МОДЕЛЬ

Главная проблема: одинаковый синтаксис скрывает разную семантику хранения

`[N]T`	**сами элементы**	копирование value копирует весь массив
`[]T`	**pointer + len + cap**	descriptor копируется, backing array может быть общим
`map[K]V`	**reference-like value**	операции идут в runtime hash table
`interface`	**type/itab + data**	есть static и dynamic type
`chan T`	**ссылка на hchan**	буфер + wait queues + lock
`Context`	**interface на цепочку объектов**	отмена распространяется по дереву

Каждую конструкцию доводим до пяти инженерных ответов

1 **Что хранится** в значении Go?

→

2 **Какая структура** стоит за ним?

→

3 **Где быстрый и** медленный путь?

→

4 **Что ограничит** рост нагрузки?

→

5 **Как увидеть** проблему?

Цена конструкции = память + CPU + синхронизация + время жизни + поведение при перегрузке

Array: длина входит в тип, value содержит элементы

```
var a [4]int
var b [5]int

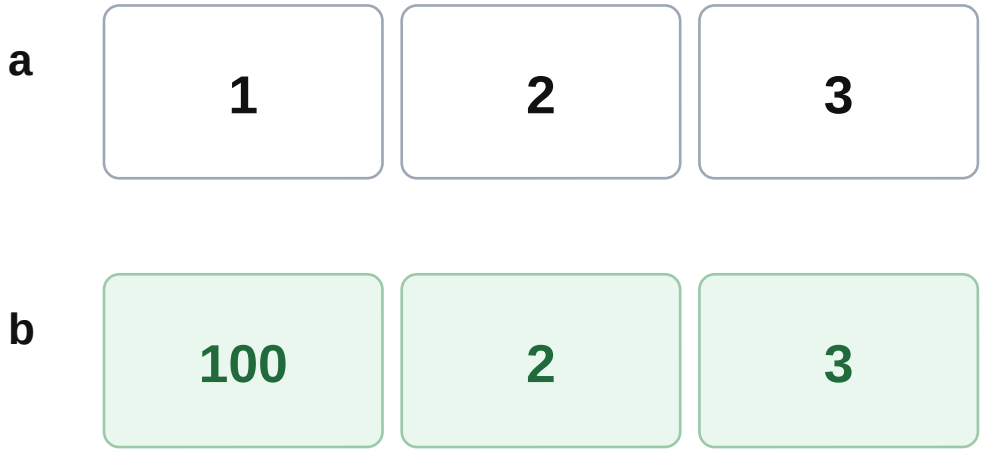
// [4]int и [5]int — разные типы
```



- Array — self-contained value: элементы являются частью значения.
- Семантика языка фиксирована: присваивание и передача параметра копируют array value.
- Компилятор может оптимизировать машинный код, но модель языка от этого не меняется.

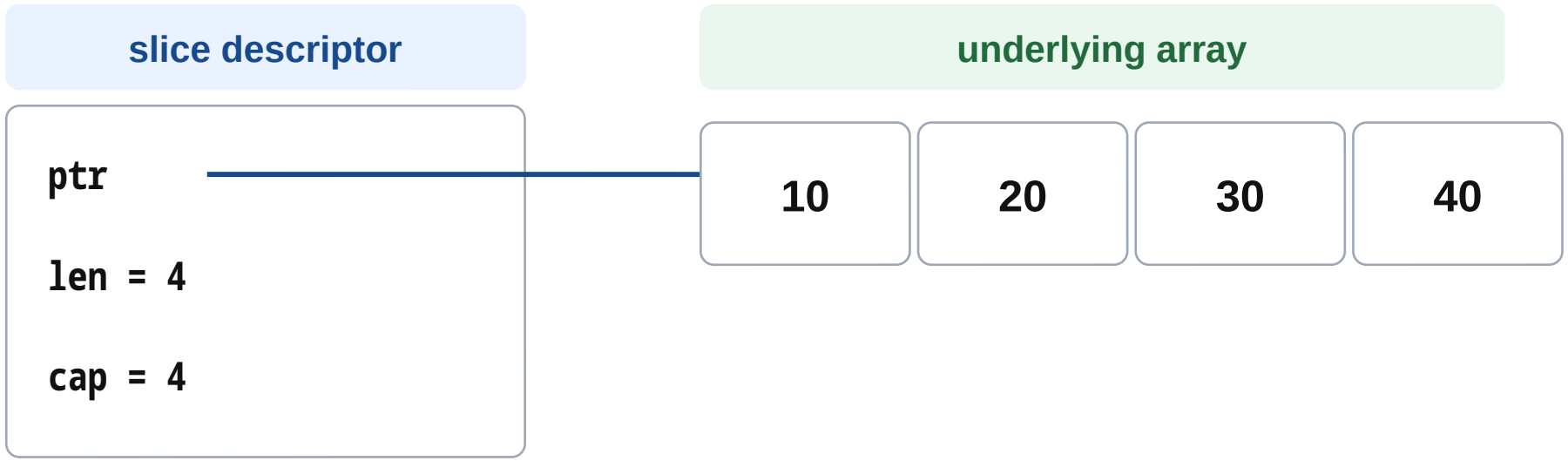
Копирование array создает независимые данные

```
a := [3]int{1, 2, 3}
b := a
b[0] = 100
```



Большой array как параметр имеет value semantics. Поэтому в API обычно передают slice или pointer, когда копия не нужна.

Slice: descriptor участка backing array



- Slice value не содержит сами элементы.
- Internal ABI на 64-bit платформе — три машинных слова: pointer, len, cap.
- Копируется descriptor; данные могут оставаться общими.

Два slice могут смотреть в один backing array

```
s := []int{10, 20, 30, 40}
a := s
a[0] = 999
```



Фраза «slice передается по значению» верна только если помнить: по значению передается descriptor, а не backing array.

len и cap описывают разные границы

```
a := [6]int{10, 20, 30, 40, 50, 60}
s := a[1:4]
```



cap считается от текущего ptr до конца доступной части backing array

Reslicing меняет descriptor, если новая длина укладывается в cap

```
s := a[1:4]   // len=3 cap=5
s = s[:5]      // len=5 cap=5
```

до



после



Новой памяти нет: меняется только len в descriptor.

append: fast path — свободная capacity есть

```
s = append(s, 42)
```

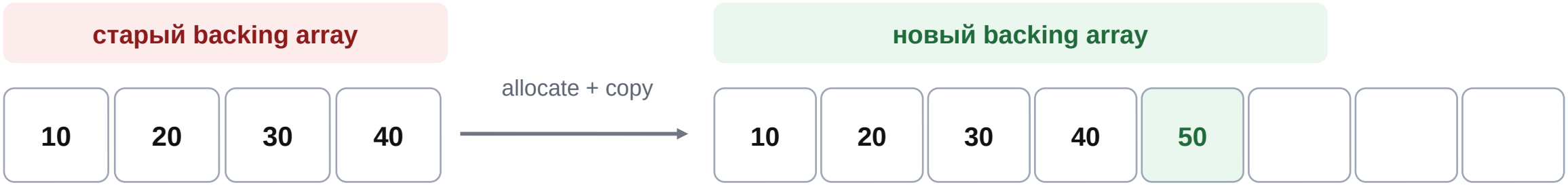
len = 3, cap = 8



- Запись идет в тот же backing array.
- ptr не меняется.
- len увеличивается, cap остается прежним.

append: slow path — capacity закончилась

```
s = append(s, 50)
```



- `runtime.growslice` вычисляет новую capacity.
- Выделяется новый backing store и копируются старые элементы.
- `append` возвращает новый descriptor — поэтому результат всегда нужно присваивать.

Рост cap — implementation detail, а не контракт языка

Устаревшее упрощение

«slice всегда удваивается»

```
oldCap := cap(s)
s = append(s, x)
newCap := cap(s)

// нельзя строить логику на
// newCap == oldCap*2
```

Текущая модель Go 1.27

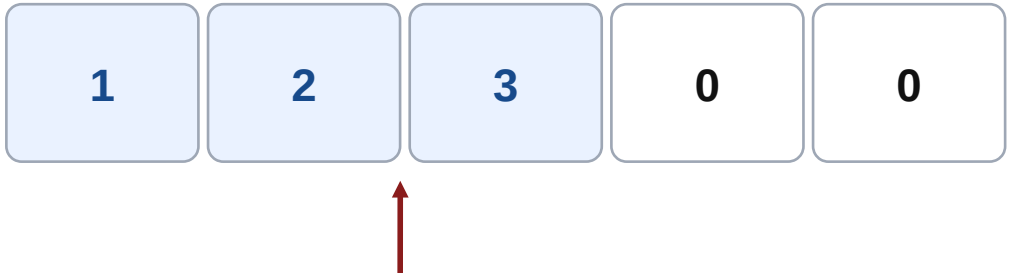
- для небольших cap — близко к удвоению;
- после порога 256 — плавный переход примерно к 1.25×;
- если сразу нужно больше — cap может прыгнуть прямо к требуемому размеру;
- allocator еще округляет фактический размер.

Гарантируется семантика append. Конкретная стратегия роста может меняться между версиями Go.

Aliasing: append одного slice может изменить другой

```
a := make([]int, 3, 5)
a[0], a[1], a[2] = 1, 2, 3
b := a[:2]
b = append(b, 100)
```

общий backing array



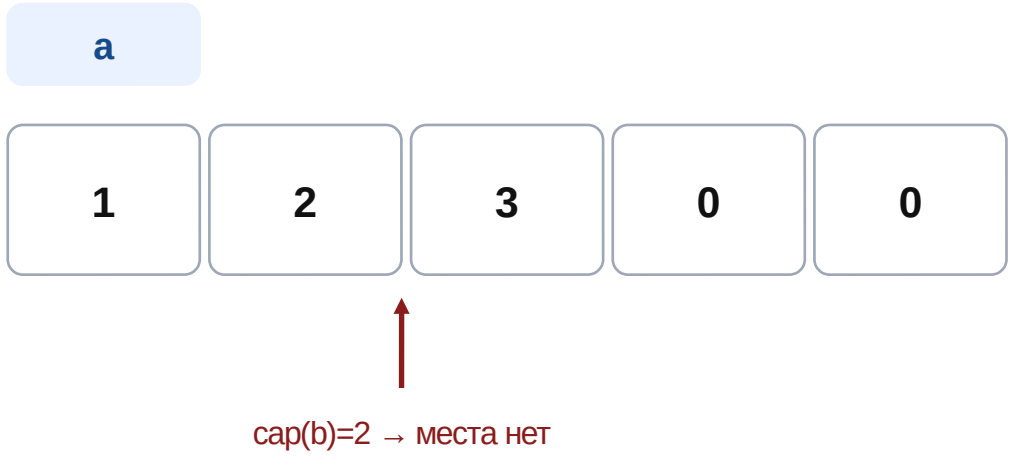
после append(b, 100)

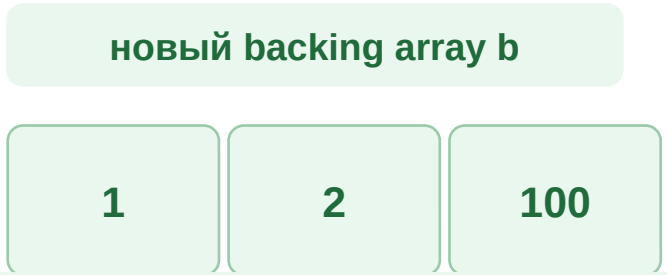


a теперь видит [1 2 100]. Capacity b хватило, поэтому realloc не произошло.

Full slice expression ограничивает право на append в общий массив

```
b := a[:2:2]   // len=2 cap=2
b = append(b, 100)
```





Полезно, когда отдаем subslice наружу и хотим запретить случайную запись за его логической границей.

Маленький subslice способен удерживать огромный allocation

```
data := make([]byte, 100<<20) // 100 MB
header := data[:10]
return header
```



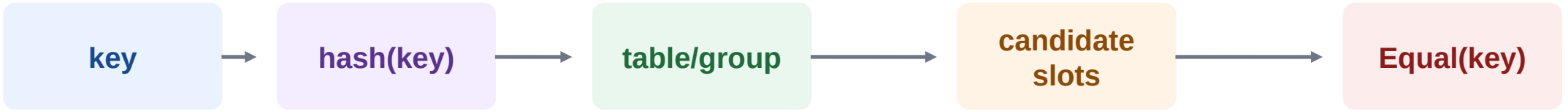
- GC видит ссылку на backing allocation целиком.
- Zero-copy уменьшает копирование, но может увеличивать retained memory.
- Если нужна независимая маленькая копия — `slices.Clone` или явный `copy`.

Slice: инженерная модель

descriptor	pointer + len + cap
copy	копируется descriptor
append	может остаться в том же массиве или сделать realloc
aliasing	разные slices могут менять одни данные
memory	subslice способен удерживать большой allocation
diagnostics	heap profile + alloc profile + benchmark

Слайс выглядит как контейнер. В production его стоимость определяется backing array, копированием и lifetime.

Map: lookup начинается не с перебора keys, а с hash



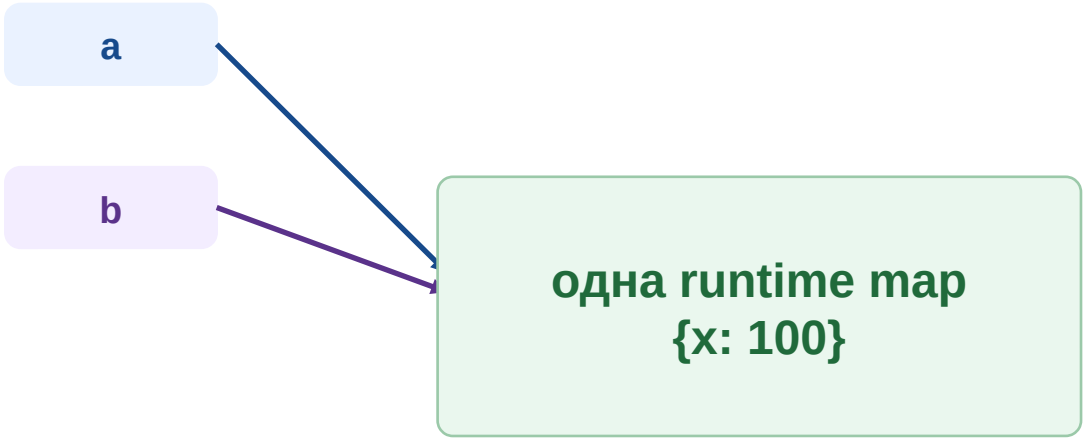
Среднее O(1) ничего не говорит о константах

- hash computation
- cache misses
- collision handling
- key comparison
- memory locality

Map value — reference-like; nil map читается, но не принимает записи

```
a := map[string]int{"x": 1}
b := a
b["x"] = 100

fmt.Println(a["x"]) // 100
```




```
var m map[string]int

_ = m["x"]   // 0
len(m)         // 0
delete(m, "x") // ok
m["x"] = 1    // panic
```

`nil map` — валидное пустое значение для чтения. Store требует инициализированную map.

Версионная граница: builtin map изменилась в Go 1.24

до Go 1.24

```
hmap
bmap
B
tophash
overflow
oldbuckets
nevacuate
```

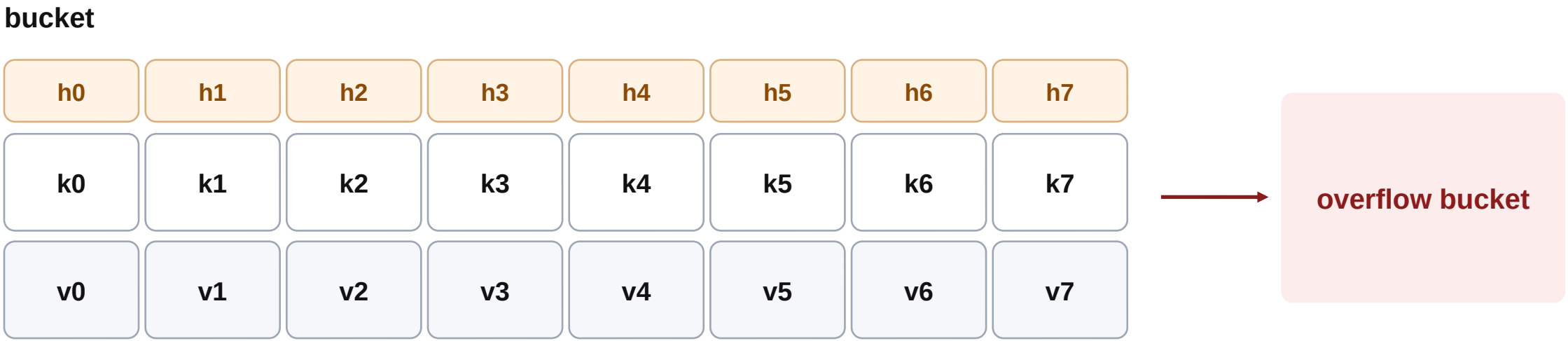


Go 1.24+

```
Swiss Tables
open addressing
control bytes
H1 / H2
probing
tombstones
directory + tables
```

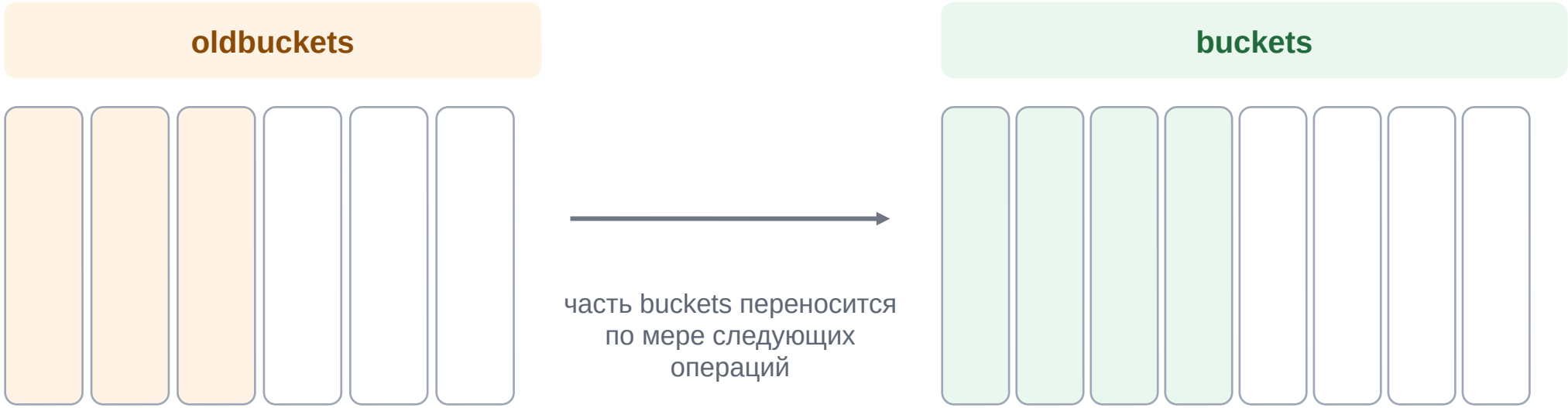
Старые статьи полезны для истории и эволюции, но `hmap/bmap` больше не описывает текущий builtin map.

Старая map: bucket на 8 slots + overflow chain



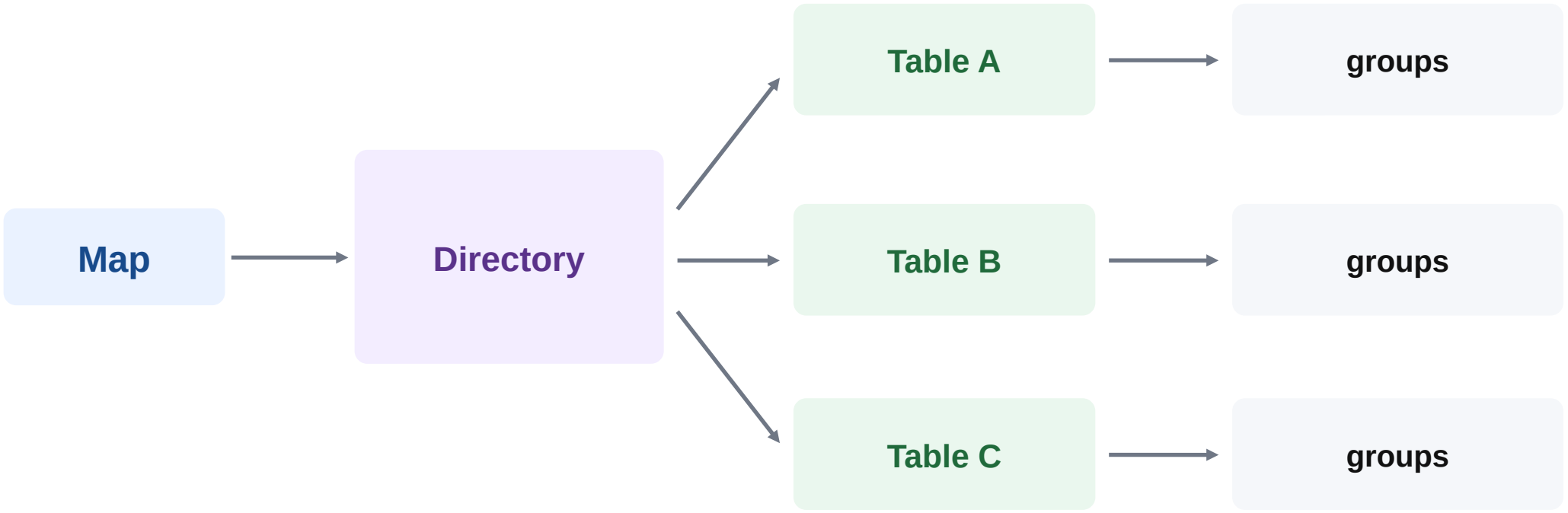
- часть hash выбирает bucket
- tophash отсекает часть кандидатов
- overflow добавляет pointer chasing и cache misses

Старая map избегала большого latency spike через incremental evacuation



- Grow не обязан переносить всю map за один Store.
- Работа амортизируется по последующим операциям.
- Это пример runtime-дизайна, где O(N) работу размазывают ради latency.

Swiss Table: groups + open addressing + compact metadata



Внутри table — probing. Directory позволяет отдельным tables расти и split-иться независимо.

Group: 8 slots и 8 control bytes

control word

12	91	53	38	53	E	7	D

slots

k0 / v0	k1 / v1	k2 / v2	k3 / v3	k4 / v4	k5 / v5	k6 / v6	k7 / v7

occupied: H2

empty

deleted

H2 — 7-bit fingerprint. Metadata позволяет быстро отфильтровать кандидатов до полного сравнения key.

Lookup: H1 наводит на область, H2 фильтрует slots

```
m["kirill"]
   ↓
hash(key)
   ↓
H1 + H2
```

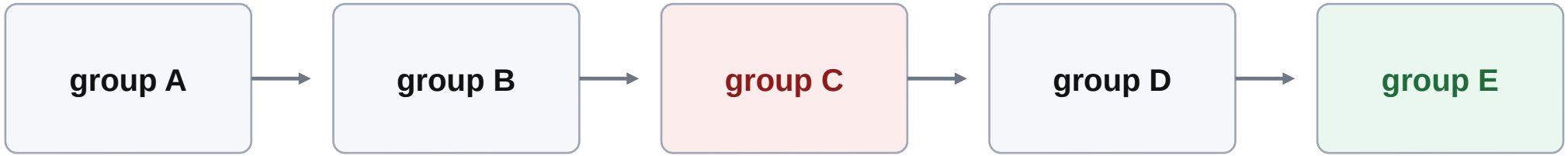
H2(search) = 53



полное Equal(key)

Совпадение H2 — только фильтр. 7 бит дают false positives; окончательное решение принимает сравнение keys.

Open addressing: collision ведет к probing, delete оставляет tombstone



probe sequence →

deleted / tombstone

не останавливает поиск

empty

может завершить поиск

Если после delete пометить slot как empty, можно оборвать probe chain и “потерять” key, лежащий дальше.

Почему Swiss быстрее: Big O тот же, стоимость memory access другая

старый дизайн

- overflow pointers
- pointer chasing
- случайные cache misses
- больше branches



Swiss

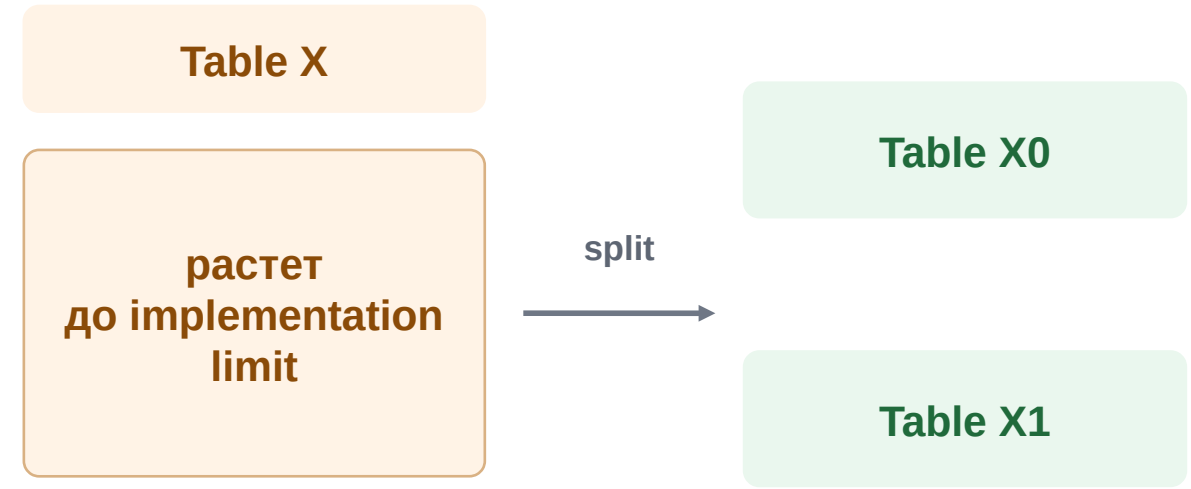
- compact metadata
- плотные groups
- меньше полных comparisons
- лучше cache locality

Старая и новая table дают lookup около O(1) в среднем. На реальном CPU выигрывают locality, меньше memory stalls и меньше лишних key comparisons.

Grow Swiss map: load factor, directory и split

load factor ≈ 7/8

Свободные slots нужны, чтобы probing не деградировал и мог завершаться.



Directory перенаправляет часть hash space в новые tables. Большая map разбивается на независимо растущие области.

make(map, hint): hint уменьшает grows, но не задает жесткую capacity

```
m := make(map[string]int, 100_000)
```

- 100_000 — ориентир initial space, а не максимальное число элементов.
- Runtime не обязан выделять ровно 100_000 slots.
- Хороший hint уменьшает количество промежуточных grow/reallocation работ.

где особенно полезно

- rows → map
- batch aggregation
- cache warmup
- JSON/result grouping
- индексы по заранее известному batch

Hint имеет смысл, когда размер действительно приблизительно известен. Угадывание “с запасом ×100” само создает memory pressure.

Обычная map не синхронизирует concurrent read/write

```
go func() {
    m["x"] = 1
}()

go func() {
    fmt.Println(m["x"])
}()
```

data race

Runtime может обнаружить некоторые опасные concurrent операции и аварийно завершить программу.

варианты

- map[K]V + Mutex/RWMutex
- ownership одной goroutine
- специализированный sync.Map

Fatal “concurrent map read and map write” — не механизм синхронизации и не замена race detector.

sync.Map тоже поменяла устройство

историческая модель

```
read
dirty
misses
entry
expunged
global-ish mutex on slow path
```

текущая Go 1.26/1.27

```
sync.Map
  ↓
internal/sync.HashTrieMap[any, any]

16 children / node
4 bits hash per level
```

read/dirty/misses важно знать как историю дизайна, но не как устройство текущего sync.Map.

Исторический sync.Map: read fast path + dirty slow path

read → **atomic / fast reads**

dirty → **lookup/store под lock**

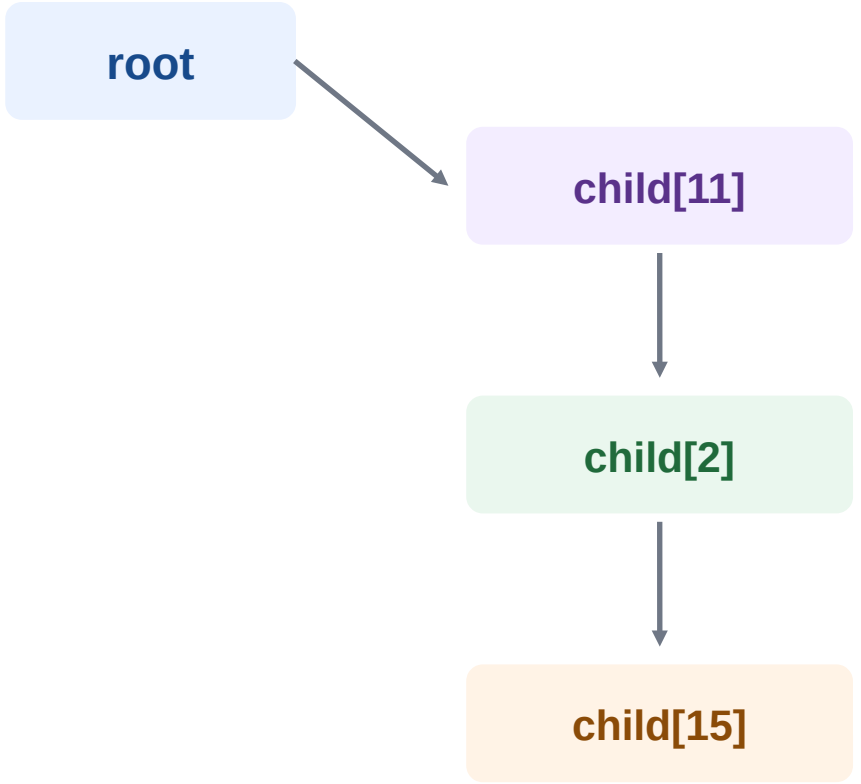
misses → **слишком много misses → dirty продвигается в read**

Адаптивный дизайн: дорогое переустройство выполнялось тогда, когда slow path уже накопил достаточную цену.

Текущий sync.Map: HashTrieMap делит hash по 4 bits на уровень

```
hash = 1011 0010 1111 0100 ...
       └──┘ └──┘ └──┘
        11    2    15
```

- Внутренний node имеет 16 children.
- Lookup идет по atomic child pointers.
- Полный hash collision может уйти в overflow list у leaf.



HashTrieMap: reads lock-free-ish по структуре, writes блокируют локальный node

Load

- hash
- atomic child load
- next level
- entry

Store

- hash + locate node
- lock local node
- re-check state
- insert / expand
- unlock

Granular synchronization уменьшает contention, если goroutines работают с разными частями trie.

sync.Map — специализированный инструмент, а не “map, но быстрее”

подходит

- read-mostly / write-once entries
- goroutines работают преимущественно с разными keys
- простые atomic operations над одной entry

map + Mutex/RWMutex

- typed API
- сложный invariant из нескольких структур
- compound update под одним lock
- предсказуемая ownership model

sync.Map синхронизирует операции над map. Ваш бизнес-invariant она автоматически атомарным не делает.

```
mu.Lock()
m1[id] = ...
m2[id] = ...
counter++
mu.Unlock()
```

Interface: dynamic type information + data representation

```
var x any = 42
```

empty interface (any)

```
Type *Type

Data unsafe.Pointer
```

non-empty interface

```
ITab *ITab

Data unsafe.Pointer
```

Это implementation/ABI модель текущего Go. На уровне языка важно: interface несет dynamic type и dynamic value.

Static type и dynamic type — разные уровни

```
func printValue(v any) {
    // static type: any
}

var x any = 42
```

static type

any

dynamic type

int

dynamic value

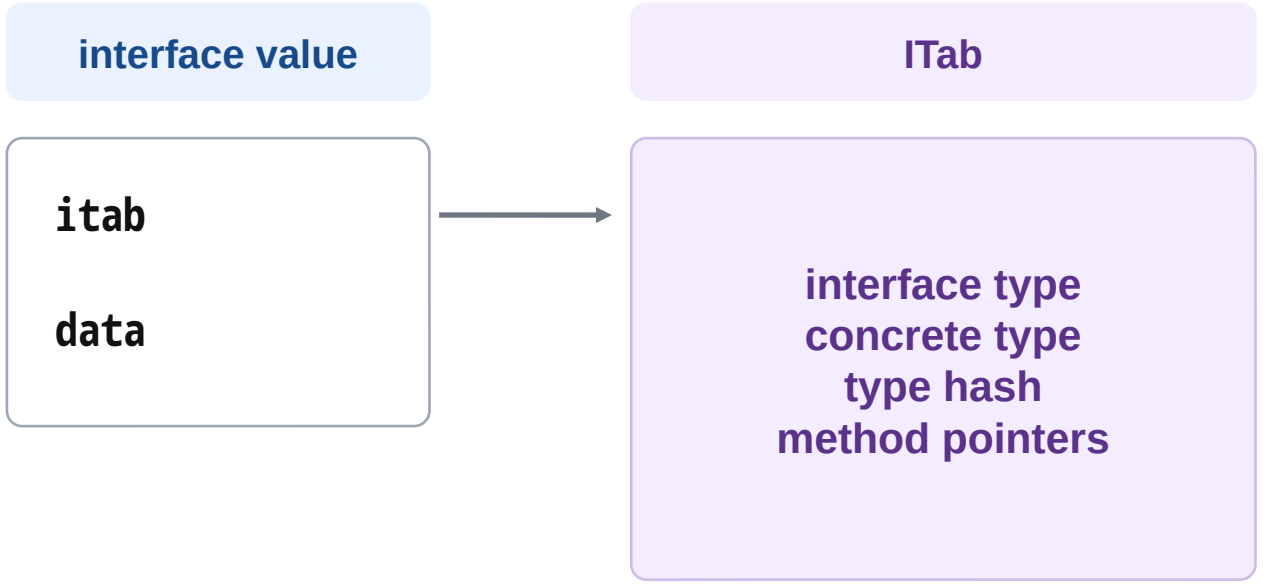
42

Type assertion и interface dispatch работают с runtime type information, а не только со static type переменной.

Non-empty interface: ITab связывает interface type, concrete type и методы

```
type Writer interface {
    Write([]byte) (int, error)
}

var w Writer = f
```



Вызов `w.Write(...)` использует подготовленную информацию о том, как concrete type реализует конкретный interface.

Typed nil: interface может содержать nil value и все равно быть != nil

```
type MyError struct{}
func (*MyError) Error() string { return "error" }

var e *MyError = nil
var err error = e

fmt.Println(err == nil) // false
```

nil interface

```
(type=nil, data=nil)
```

typed nil inside interface

```
(type=*MyError, data=nil)
```

Проверка `interface == nil` истинна только когда отсутствуют и dynamic type, и dynamic value.

Type assertion спрашивает runtime о dynamic type

```
s, ok := v.(string)

// ok=false, panic нет
```

concrete assertion

проверка runtime type information

```
s := v.(string)

// несовместимый dynamic type
// → panic
```

assertion к interface

нужно проверить, реализует ли concrete type нужный interface; ITab может кэшироваться

Boundary code обычно использует форму с `ok`, если несовпадение типов является ожидаемым состоянием.

Interface conversion не означает автоматическую heap allocation

на стоимость влияют

- concrete type
- representation
- escape analysis
- lifetime
- compiler optimizations

```
go build -gcflags="-m=2"

go test -bench=. -benchmem
```

не угадывать — измерять

“Положили value в interface → обязательно allocation” — плохая модель. Конкретный результат зависит от escape analysis и ABI representation.

Channel: synchronization object, а не просто “потокобезопасная очередь”

```
ch := make(chan int, 4)
```

chan value

hchan

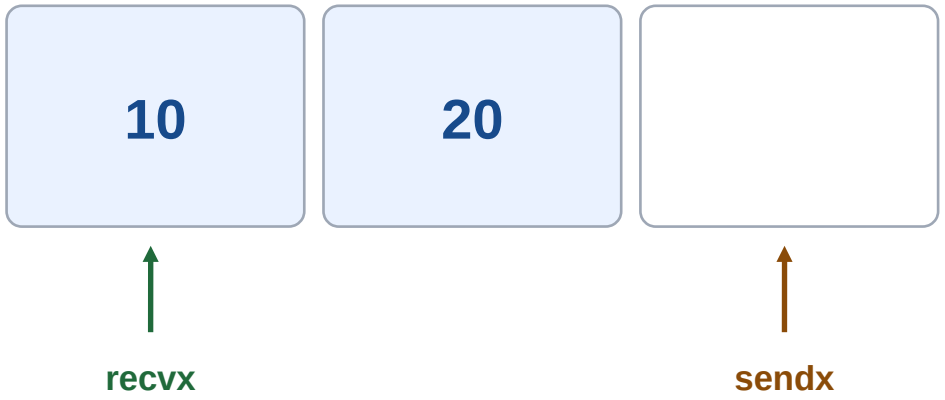
```
qcount
dataqsiz
buf
sendx / recvx
sendq / recvq
closed
lock
```

ring buffer
(if buffered)

waiting goroutines
via sudog queues

Buffered channel: circular buffer + send/receive indices

```
ch := make(chan int, 3)
```



qcount = 2

dataqsiz = 3

Индексы закольцовываются. Channel buffer — ограниченная очередь, размер которой влияет на backpressure и latency.

Send/receive имеют несколько путей выполнения

send ch <- v

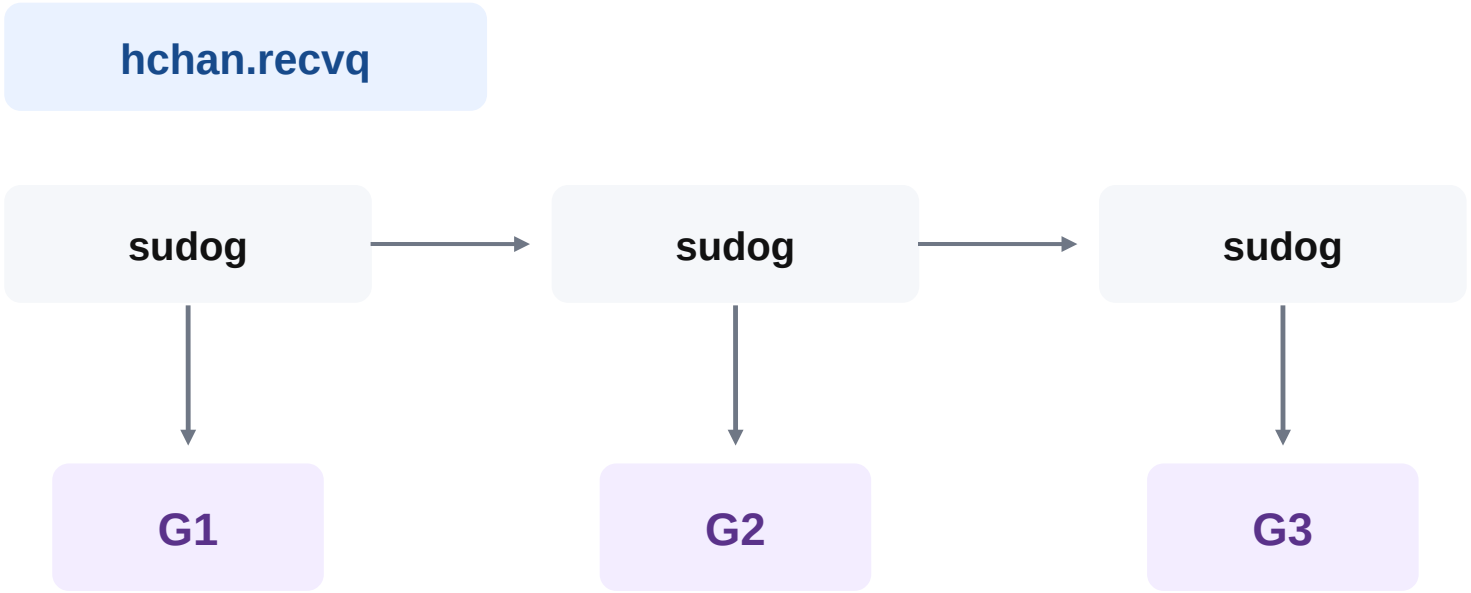
- есть waiting receiver → согласовать напрямую
- receiver нет, buffer не full → положить в ring buffer
- buffer full → sender в sendq и park

receive v := <-ch

- есть waiting sender → согласовать
- есть buffered data → взять по recvx
- данных нет → receiver в recvq и park

Channel operation может быть дешевой передачей, обращением к buffer или полноценным scheduler interaction.

sudog связывает goroutine с очередью ожидания channel



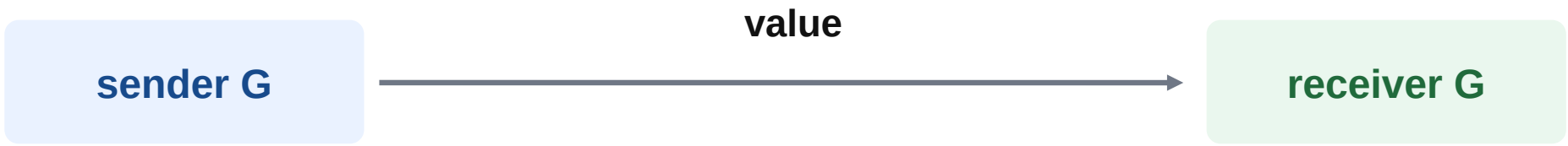
wake path

```
dequeue sudog
     ↓
goready(g)
     ↓
G runnable
```

Блокируется goroutine, а не обязательно OS thread. Scheduler может использовать M для другой runnable G.

Unbuffered channel — rendezvous sender и receiver

```
ch := make(chan int) // cap = 0
```



нет пользовательского ring buffer

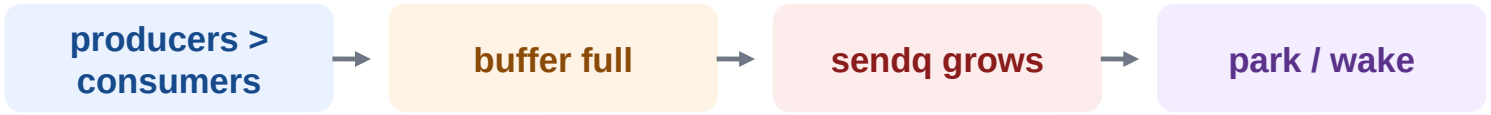
- send завершается только при наличии receiver
- receive — при наличии sender
- runtime может передавать значение напрямую между goroutines

Под нагрузкой channel становится точкой backpressure и contention

```
jobs := make(chan Job, 100)

1000 producers
10 workers
```

hot hchan lock + cache-line bouncing + scheduler work



Buffer size — архитектурный параметр. Он определяет, сколько нагрузки система временно поглотит до блокировки producers.

close и nil channel задают две разные специальные семантики

closed channel

- новый send → panic
- buffered values можно дочитать
- после опустошения receive → zero value, ok=false
- close пробуждает ожидающие goroutines

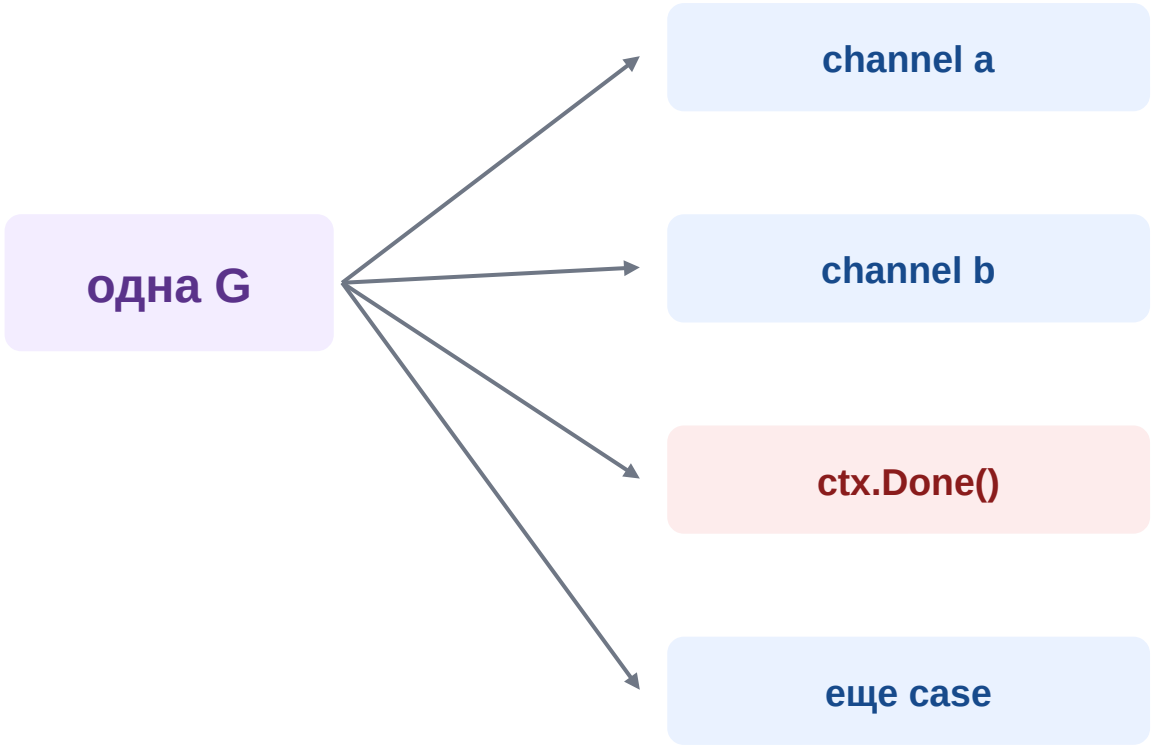
nil channel

- send ждет бесконечно
- receive ждет бесконечно
- в select такой case никогда не ready

Nil channel кажется бесполезным до тех пор, пока не начинаем динамически включать и выключать cases в select.

select: одна goroutine должна ждать сразу несколько channels

```
select {
case v := <-a:
    use(v)
case b <- result:
    sent()
case <-ctx.Done():
    cancel()
}
```



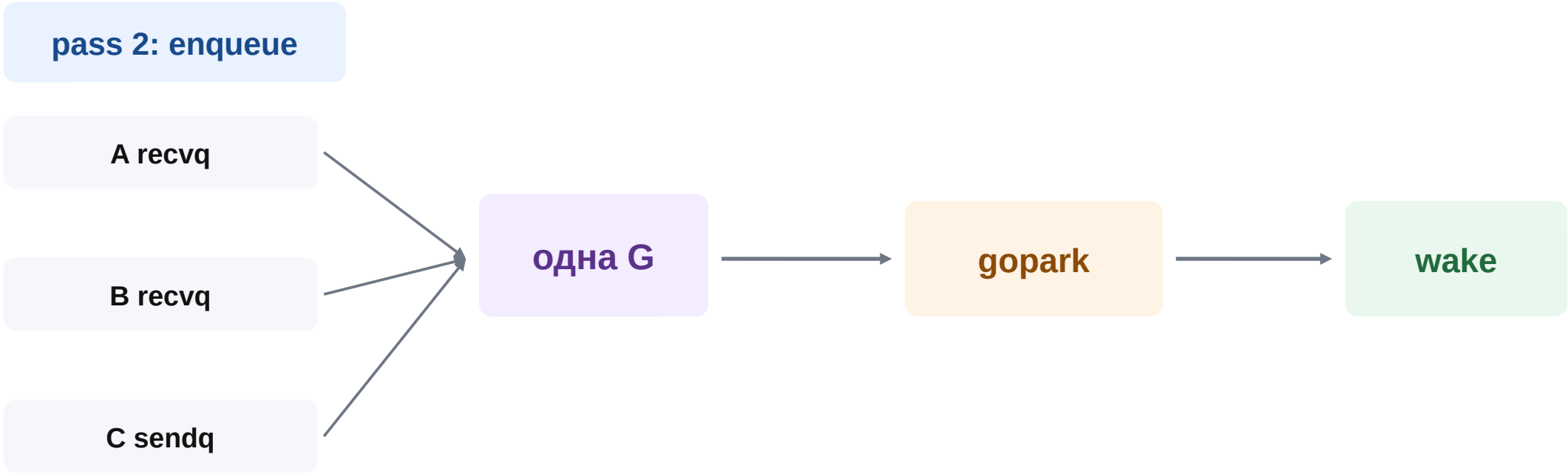
G не может “gopark четыре раза”. Runtime должен зарегистрировать одно ожидание сразу во всех wait queues.

selectgo: randomized poll order + deterministic lock order

1. poll order	2. lock order	3. ready scan
cases перемешиваются, чтобы верхний case не получал систематический приоритет	channels сортируются по адресу; locks берутся в одном порядке	sender? receiver? buffered data? space? closed?

Если ready case найден — выполняем. Если нет и есть default — возвращаемся сразу. Иначе начинается регистрация ожидания.

selectgo: enqueue → park → wake → cleanup проигравших cases



После wake runtime повторно lock-ит channels и удаляет sudog из всех проигравших очередей. Выиграл один case — registrations было несколько.

select edge cases: nil case, default и fairness

```
var ch <-chan int

select {
case v := <-ch: // ch=nil → disabled
    _ = v
case <-ctx.Done():
}
```

nil case никогда не ready

```
for {
    select {
    case v := <-ch:
        process(v)
    default:
    }
}
```

default может превратить loop в busy polling → 100% CPU

Выбор среди одновременно ready cases pseudo-random; strict fairness и priority policy нужно реализовывать явно.

Context: обычный interface, который переносит lifetime операции

```
type Context interface {
    Deadline() (time.Time, bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

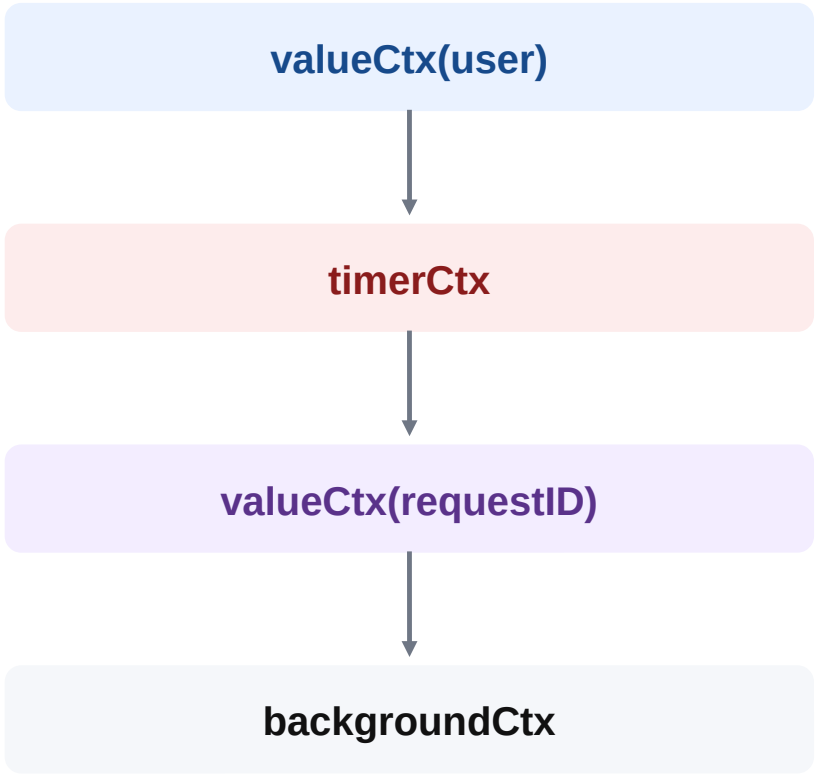
что он переносит

- deadline
- cancellation signal
- request-scoped values
- связь parent → child

Context не “убивает goroutine”. Cancellation cooperative: код или underlying API должны наблюдать Done/Err.

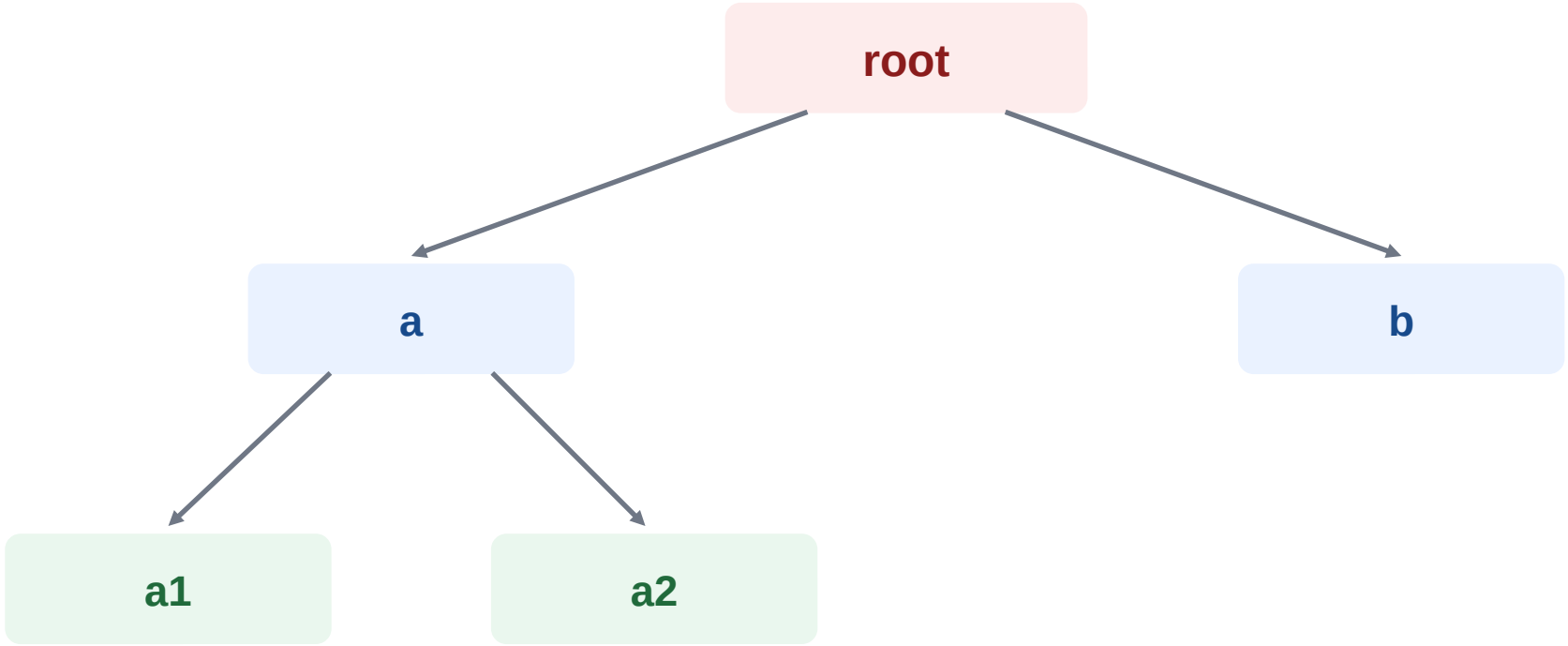
Derived contexts образуют цепочку wrapper-объектов

```
ctx := context.Background()
ctx = context.WithValue(
    ctx, requestIDKey, requestID)
ctx, cancel := context.WithTimeout(
    ctx, time.Second)
ctx = context.WithValue(ctx, userKey, user)
```



`Context.Value` ищет key по цепочке. Это не обычная hash map и не универсальный dependency container.

Cancellation образует дерево parent → descendants

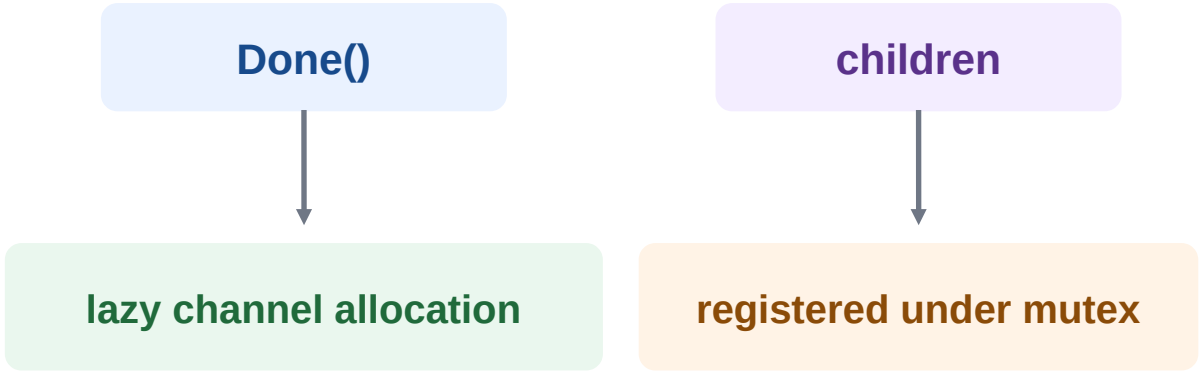


cancelRoot()

Отмена родителя распространяется вниз. Child не может пережить parent cancellation без явного detach.

cancelCtx хранит children и лениво создает Done channel

```
type cancelCtx struct {
    Context
    mu       sync.Mutex
    done     atomic.Value
    children map[canceler]struct{}
    err      atomic.Value
    cause    error
}
```



Если Done никогда не вызывается, отдельный channel можно вообще не создавать — маленькая, но важная allocation optimization.

cancel(): закрыть Done, пройти children, распространить причину

1. set err/cause → 2. close(done) → 3. cancel children → 4. clear children → 5. detach from parent

почему close(done)?

close делает receive ready для всех текущих и будущих receivers → broadcast без N отдельных send.

scheduler effect

много blocked goroutines могут одновременно стать runnable

`cancel()` подает сигнал — работа должна остановиться сама

Ошибка: сигнал игнорируется

```
go func() {
    for {
        doCPUWork()
    }
}()
```

Код обрабатывает отмену

```
go func() {
    for {
        select {
        case <-ctx.Done(): return
        default: doCPUWork()
        }
    }
}()
```

`cancel()` не прерывает бесконечный цикл, который не проверяет `Done`.

WithTimeout = cancellation + timer; CancelFunc освобождает ресурсы раньше

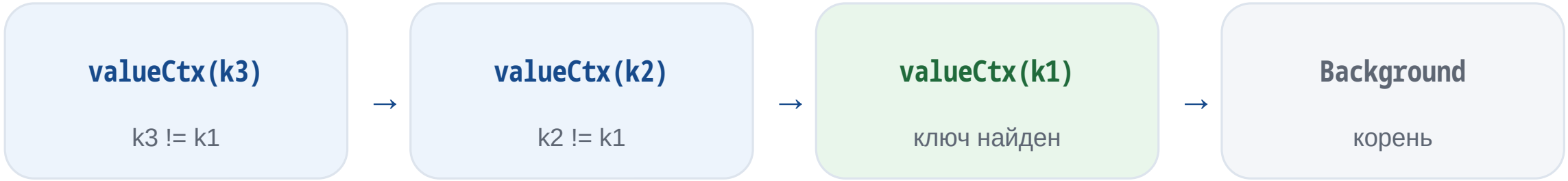
```
ctx, cancel := context.WithTimeout(
    parent, 30*time.Second)
defer cancel()
```

работа закончилась за 20 ms → **cancel()** → **timer/child linkage можно освободить**

Если cancel забыть, child/timer могут жить до parent cancellation или deadline. При большом RPS это превращается в лишнее retained state.

`Context.Value` проходит цепочку оберток — это не `map`

```
ctx := context.WithValue(parent, userKey, user)
```



Поиск идет вверх по цепочке до совпадения ключа. Чем длиннее путь, тем больше сравнений.

`WithValue` предназначен для метаданных запроса, а не как универсальный контейнер зависимостей.

Custom Context иногда заставляет propagation использовать отдельную goroutine

обычный parent cancelCtx

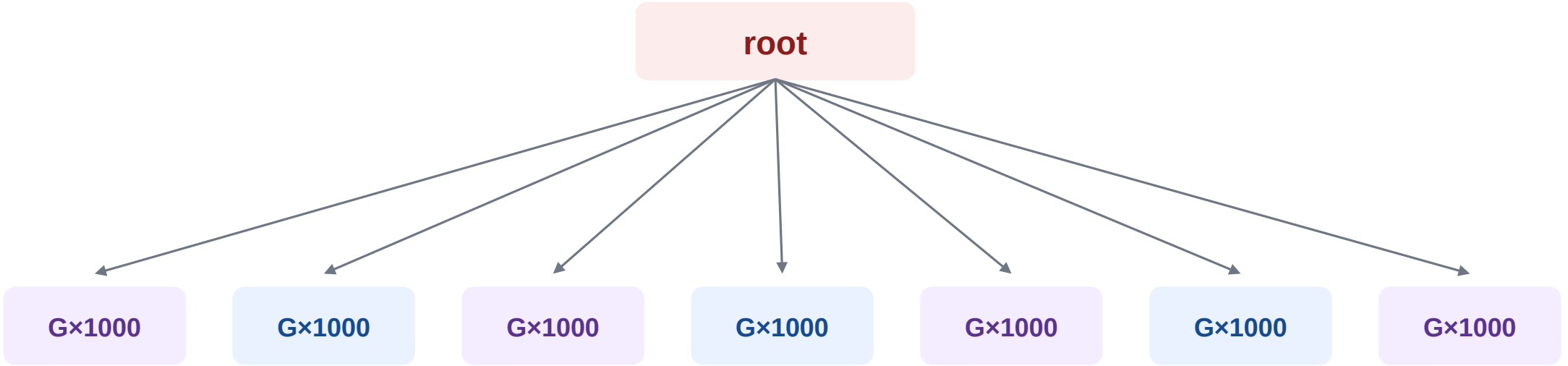
child регистрируется напрямую в parent.children

fallback для custom Context

```
go func() {
    select {
    case <-parent.Done():
        child.cancel(...)
    case <-child.Done():
    }
}()
```

Знание internals полезно: своя реализация Context может неожиданно изменить стоимость обычного WithCancel.

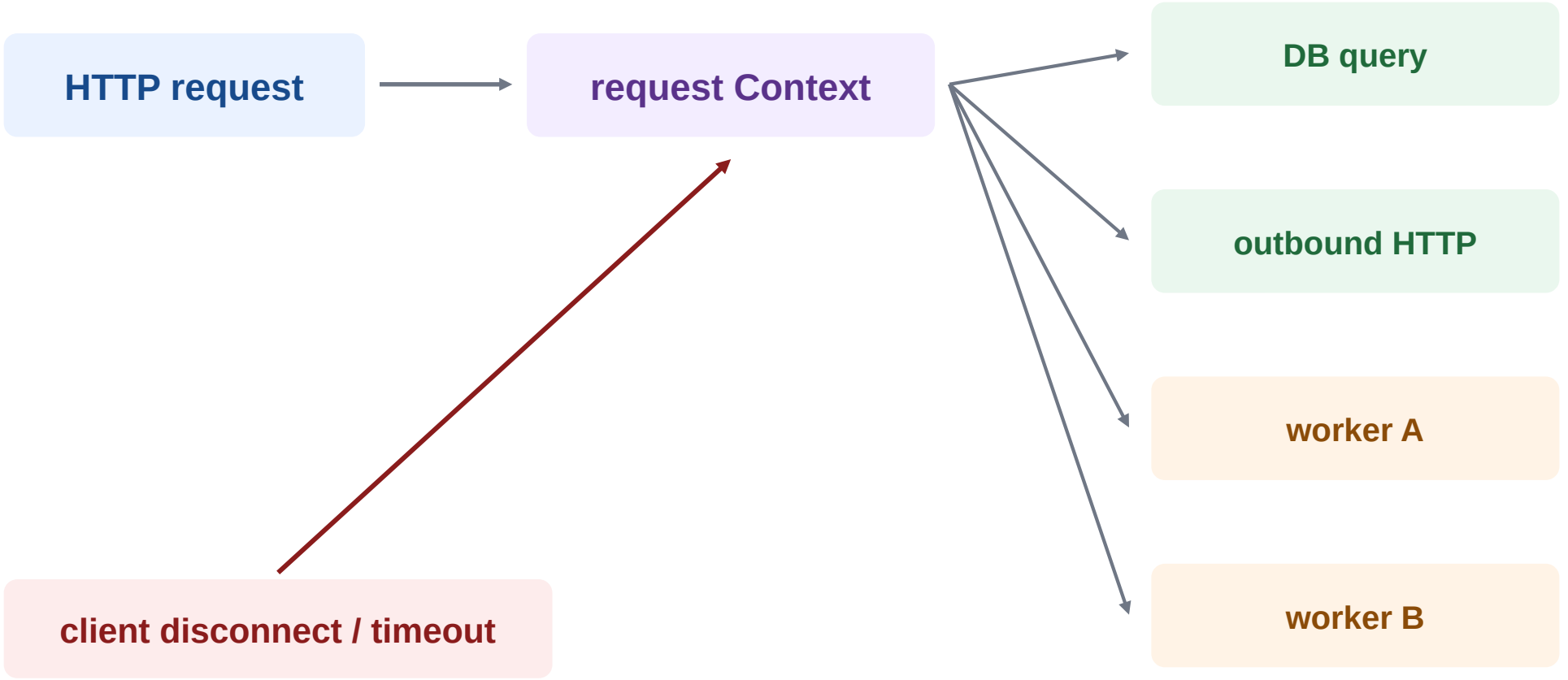
Cancellation storm: один cancel может разбудить тысячи goroutines



cancel root → close Done → children cancel → G runnable → scheduler burst

Логическая операция cancel дешевая по API, но физически может породить заметный scheduler burst под нагрузкой.

HTTP backend: context переносит lifetime запроса вниз по стеку



Нормальная propagation прекращает бесполезную downstream работу и раньше освобождает pool/CPU/network resources.

Как ломают context: detach, игнорирование Done и “расширение” deadline

```
func handler(w http.ResponseWriter,
    r *http.Request) {
    go process(context.Background())
}
```

detach

request закончился, background work продолжает жить

```
ctx, cancel := context.WithTimeout(
    r.Context(), 10*time.Second)
```

parent deadline = 2s

effective lifetime ≈ 2s

- goroutine не проверяет Done → cancel ничего не завершает
- забытый CancelFunc → лишние timers/children
- Background внутри request path разрывает lifetime chain

Отмена управляет временем жизни, но не ограничивает поток работы

Что дает `Context`

- тайм-аут операции
- сигнал отмены
- раннее освобождение ресурсов
- отмена дочерних вызовов

Чего он не гарантирует

- предел одновременных запросов
- ограничение очереди
- контроль скорости поступления
- автоматическую остановку чужого кода

Отмена предотвращает работу, результат которой уже не нужен, но лимиты нагрузки нужны отдельно.

ИТОГ

Одна строка Go может тянуть за собой весь runtime stack

append	**growslice → allocation → copy → heap → GC**
map lookup	**hash → directory → group → metadata → Equal → cache lines**
channel send	**hchan lock → buffer/sendq → sudog → gopark → scheduler**
cancel	**close Done → descendants → goroutines runnable → scheduler burst**

Цель internals — не запоминать названия структур runtime. Цель — уметь объяснить, почему конкретный код меняет память, latency и поведение под нагрузкой.

Диагностика: от симптома к runtime-механизму

heap растет	slice retention / allocations	heap pprof, allocs, `-m=2`
CPU растет	hashing / busy select / GC	CPU pprof, trace
goroutines растут	chan send/receive/select / lost cancellation	goroutine profile
latency хвосты	channel contention / parking / wakeups	block profile, trace
работа живет после HTTP request	сломана context propagation	goroutine dump + code path

Семантика Go стабильнее, чем внутренние структуры рантайма

Язык / публичный API

- копирование массива и среза
- семантика `len`, `cap`, `append`
- `map` и typed `nil`
- отправка / получение / `close`
- правила `select` и `context`

Текущая реализация Go

- стратегия роста `cap`
- Swiss Tables / HashTrieMap
- `hchan`, `sudog`, `selectgo`
- поля `cancelCtx`
- ленивый канал `Done`

Версионные механизмы нужны для диагностики, но прикладная логика не должна требовать их неизменности.

ПРОДОЛЖЕНИЕ КУРСА

Следующие лекции открывают алгоритмы, а не повторяют основы

4.1 · Структуры глубже

growslice · Swiss Tables · HashTrieMap

ABI интерфейсов

4.2 · Channels / select / context

chansend · chanrecv · selectgo

propagateCancel

4.3 · Компилятор Go

SSA · lowering · оптимизации

fast path / runtime helper / asm

Граница базовой лекции №4: правильная модель, стоимость, нагрузка и диагностика.

ИТОГ

Что важно запомнить

- Array — value с собственными элементами; slice — descriptor над backing array.
- `append` может менять общий массив или перевести slice на новый allocation.
- Builtin map с Go 1.24 — Swiss Table; pre-1.24 hmap/bmap — историческая модель.
- Текущий `sync.Map` использует HashTrieMap; старые read/dirty/misses — тоже история.
- Interface хранит dynamic type information; typed nil внутри interface обычно != nil.
- Channel связывает buffer, wait queues, sudog и scheduler; hot channel может стать hotspot.
- `select` регистрирует одну goroutine в нескольких channel queues и потом чистит проигравшие registrations.
- Context — cooperative cancellation tree поверх interfaces, channels, mutex/atomic и timers.

Главная формула: syntax → representation → mechanism → load → diagnostics